

Name: Matthew Woo

CS240 Assignment 1

GitHub Repository: <https://github.com/matthewwoo527/cs240-assignment-1>

[README.md](#)

CS240 Assignment 1

Author: Matthew Woo

Date: Sep 3, 2026

[GitHub Repository URL](#)

AI Disclose usage: I asked ChatGPT what to change in the code, and learned what is the difference between signed and unsigned 8 bits. So it can pass the negative two's complement case.

Converter and Pixel System

[1. Build an ASCII-to-decimal converter.](#)

A converter that converts ASCII to decimal.

Screenshot of the output: [q1test.png](#)

Example: When the user enters a string, it will print the decimal value of that string.

[2. Build a number-base converter supporting binary, decimal, octal, and hexadecimal.](#)

Source code file name: [Woo Matthew Assignment1 q2.py](#)

[Test cases \(go to section 5\)](#)

[3. Write a program that reads an image and prints its pixel values.](#)

Source code file name: [Woo Matthew Assignment1 q3.py](#)

Input file(image): [smiley.bmp](#)*

*For the image type I did not use .png because my paint app on win11 automatic use RGBA instead RGB so I use .bmp instead.

Output file(text): [output.txt](#)

I created my own version of smiley, and it is 10px * 10px. It has 3 colors.

Red (R): rgb(237, 28, 36), Black (B): rgb(0, 0, 0), Yellow (Y): rgb(255, 242, 0)

The program takes the image as an input file and outputs a txt file that contains the pixel values of the image.

[4. Write a program that consumes pixel values and creates an image.](#)

Source code file name: [Woo Matthew Assignment1 q4.py](#)

Input file(text): [input.txt](#)

Output file(image): [output.png](#)

5. Test boundary cases, including zero, the largest supported unsigned value, and at least one negative two's-complement value.

[Test cases for question 2](#)

- *Boundary cases*

```
Enter a number: 10
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 1010
```

```
Enter a number: 1010
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: b
Which base to convert to? [b/d/o/h] d
From base: binary, To base: decimal, Result: 10
```

```
Enter a number: 10
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] o
From base: decimal, To base: octal, Result: 12
Enter a number: 12
```

```
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: o
Which base to convert to? [b/d/o/h] d
From base: octal, To base: decimal, Result: 10
```

```
Enter a number: 255
```

```
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] h
From base: decimal, To base: hexadecimal, Result: ff
```

```
Enter a number: ff
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: h
Which base to convert to? [b/d/o/h] d
From base: hexadecimal, To base: decimal, Result: 255
```

```
Enter a number: 1010
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: b
Which base to convert to? [b/d/o/h] h
From base: binary, To base: hexadecimal, Result: a
```

```
Enter a number: a
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: h
Which base to convert to? [b/d/o/h] b
From base: hexadecimal, To base: binary, Result: 1010
```

- *Including zero*

```
Enter a number: 0
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 0
```

- *The largest supported unsigned value*

```
Enter a number: 255
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 11111111
```

```
Enter a number: 255
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] h
From base: decimal, To base: hexadecimal, Result: ff
```

- *Negative two's-complement value*

```
Enter a number: -5
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 11111011
```

```
Enter a number: -128
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 10000000
```

- *Cases that outside of 8-bits*

```
Enter a number: -129
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
Number is not in 8 bits range
```

```
Enter a number: 256
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
Number is not in 8 bits range
```

[Woo_Matthew_Assignment1_q1.py](#)

#Build an ASCII-to-decimal converter.

import sys

args = sys.argv

print(args)

s = "Dominic"

s = input("Enter a ASCII word to convert to decimal: ")

#standard

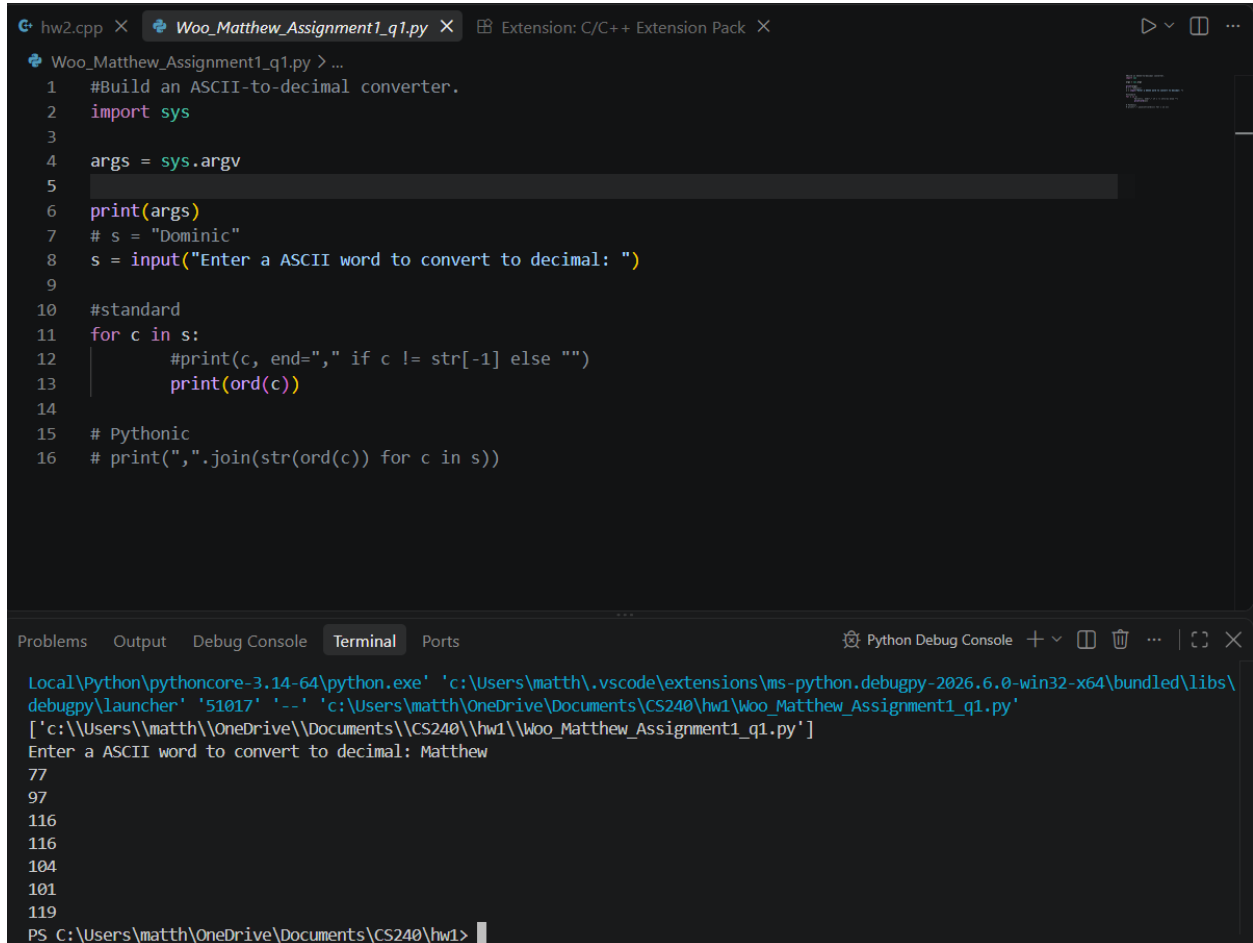
for c in s:

#print(c, end="," if c != str[-1] else "")

print(ord(c))

```
# Pythonic
# print(",".join(str(ord(c)) for c in s))
```

Test 1:



The screenshot shows a Visual Studio Code editor with a Python file named `Woo_Matthew_Assignment1_q1.py`. The code is a script that takes a command-line argument and prints its ASCII values. The terminal output shows the script being executed with the argument `Matthew`, and the output is the ASCII values of each character in the word, separated by commas.

```
Woo_Matthew_Assignment1_q1.py > ...
1 #Build an ASCII-to-decimal converter.
2 import sys
3
4 args = sys.argv
5
6 print(args)
7 # s = "Dominic"
8 s = input("Enter a ASCII word to convert to decimal: ")
9
10 #standard
11 for c in s:
12     #print(c, end=", " if c != str[-1] else "")
13     print(ord(c))
14
15 # Pythonic
16 # print(",".join(str(ord(c)) for c in s))
```

Terminal Output:

```
Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\matth\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\
debugpy\launcher' '51017' '--' 'c:\Users\matth\OneDrive\Documents\CS240\hw1\Woo_Matthew_Assignment1_q1.py'
['c:\\Users\\matth\\OneDrive\\Documents\\CS240\\hw1\\Woo_Matthew_Assignment1_q1.py']
Enter a ASCII word to convert to decimal: Matthew
77
97
116
116
104
101
119
PS C:\Users\matth\OneDrive\Documents\CS240\hw1>
```

Woo_Matthew_Assignment1_q2.py

```
#A number-base converter

#AI Disclose usage:
#I asked ChatGPT what to change in the code,
#and learned what is the difference between
#signed and unsigned 8 bits.
#So it can pass the negative two's complement case.

#Ask for number and which base this is
number = input("Enter a number: ")
convert_from_base = input("Which base is this number? Binary, decimal, octal, or
hexadecimal [b/d/o/h]: ")
convert_to_base = input("Which base to convert to? [b/d/o/h] ")

#Bit Size for negative two's complement
BIT_SIZE = 8

#This if statement is for printing the result, from base
print_from_base = "invalid base"
if convert_from_base == "b" or convert_from_base == "B":
    print_from_base = "binary"
elif convert_from_base == "d" or convert_from_base == "D":
    print_from_base = "decimal"
elif convert_from_base == "o" or convert_from_base == "O":
    print_from_base = "octal"
elif convert_from_base == "h" or convert_from_base == "H":
    print_from_base = "hexadecimal"
else:
    print("Invalid Base")

#This if statement is for printing the result, to base
print_to_base = "invalid base"
if convert_to_base == "b" or convert_to_base == "B":
    print_to_base = "binary"
elif convert_to_base == "d" or convert_to_base == "D":
    print_to_base = "decimal"
elif convert_to_base == "o" or convert_to_base == "O":
    print_to_base = "octal"
elif convert_to_base == "h" or convert_to_base == "H":
    print_to_base = "hexadecimal"
else:
    print("Invalid Base")

#Set the base to an int so the int function can take it as the parameter
if convert_from_base == "b" or convert_from_base == "B":
    convert_from_base = 2
```

```

elif convert_from_base == "d" or convert_from_base == "D":
    convert_from_base = 10
elif convert_from_base == "o" or convert_from_base == "O":
    convert_from_base = 8
elif convert_from_base == "h" or convert_from_base == "H":
    convert_from_base = 16
else:
    print("Invalid Base")

#It takes a str and int base and return a interger of that base
num = int(number, convert_from_base)

#AI Disclose: Inspired by ChatGPT
#It checks if the number fits 8 bits
#and convert the negative number to 8 bit two's complement
#It can handle the negative two's complement case too.

#Unsigned: 0 to 255
#Signed: -128 to 127
#This code can handle both cases

if num > 255 or num < -128: #From -128 to -1 and 0 to 255
    print("Number is not in 8 bits range")
    exit()
if num < 0:
    converted_num = num + (2 ** BIT_SIZE) #num + 2^8, turns negative decimal to two's
    complement
else:
    converted_num = num

#The convert part, convert the num to the base that the user ask for and save it to
the variable "result"
if convert_to_base == "b" or convert_to_base == "B":
    result = bin(converted_num)[2:] #The "[2:]" takes the integer after index 2, since
the integer in index 0 and 1 are the decimal type(0b, 0o, 0x)
    if num < 0: #In the case of negative number, fill zeros on the left for the result
        result = result.zfill(BIT_SIZE) #it fill zeros on the left
elif convert_to_base == "d" or convert_to_base == "D":
    result = num #Use num because num is already in decimal
elif convert_to_base == "o" or convert_to_base == "O":
    result= oct(converted_num)[2:]
elif convert_to_base == "h" or convert_to_base == "H":
    result = hex(converted_num)[2:]
else:
    print("Invalid Base")

print("From base: " + print_from_base + ", To base: " + print_to_base + ", Result: " +
str(result))

```

Test Cases for problem 2:

- *Boundary cases*

```
Enter a number: 10
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 1010
```

```
Enter a number: 1010
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: b
Which base to convert to? [b/d/o/h] d
From base: binary, To base: decimal, Result: 10
```

```
Enter a number: 10
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] o
From base: decimal, To base: octal, Result: 12
Enter a number: 12
```

```
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: o
Which base to convert to? [b/d/o/h] d
From base: octal, To base: decimal, Result: 10
```

```
Enter a number: 255
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] h
From base: decimal, To base: hexadecimal, Result: ff
```

```
Enter a number: ff
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: h
Which base to convert to? [b/d/o/h] d
From base: hexadecimal, To base: decimal, Result: 255
```

```
Enter a number: 1010
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: b
Which base to convert to? [b/d/o/h] h
From base: binary, To base: hexadecimal, Result: a
```

```
Enter a number: a
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: h
Which base to convert to? [b/d/o/h] b
From base: hexadecimal, To base: binary, Result: 1010
```

- *Including zero*

```
Enter a number: 0
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 0
```

- *The largest supported unsigned value*

```
Enter a number: 255
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 11111111
```

```
Enter a number: 255
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] h
From base: decimal, To base: hexadecimal, Result: ff
```

- *Negative two's-complement value*

```
Enter a number: -5
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 11111011
```

```
Enter a number: -128
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
From base: decimal, To base: binary, Result: 10000000
```

- *Cases that outside of 8-bits*

```
Enter a number: -129
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
Number is not in 8 bits range
```

```
Enter a number: 256
Which base is this number? Binary, decimal, octal, or hexadecimal
[b/d/o/h]: d
Which base to convert to? [b/d/o/h] b
Number is not in 8 bits range
```

Woo_Matthew_Assignment1_q3.py

```
#import Image module from Pillow Library
from PIL import Image
```

#A function called convert, that takes RGB String and convert to a shorter and easier string to read.

```
def convert(color):
    if color == "(237, 28, 36)":
        return "R"
    elif color == "(0, 0, 0)":
        return "B"
    elif color == "(255, 242, 0)":
        return "Y"
    else:
        return color
```

#Open a file called "output.txt" in w(write) mode

```
output_file = open("output.txt", "w")
```

#Open the image called "smiley.bmp" and load that image

```
image_file = Image.open("./smiley.bmp")
image_file.load()
```

#Save the image size to the variable called width and height

```
width, height = image_file.size
```

#A forloop that run through each line

```
for y in range(height):
```

 #A forloop that run through each pixel of each line

```
    for x in range(width):
```

 #Get the pixel of that box that currently at, turn it to a string type and put it in the variable called "color"

```
        color = str(image_file.getpixel((x,y)))
```

 #Call the function that we created above, put the results return from the function and save it into the variable "color"

```
        color = convert(color)
```

 #Write that color and a space " " after it

```
        output_file.write(color)
```

```
        output_file.write(" ")
```

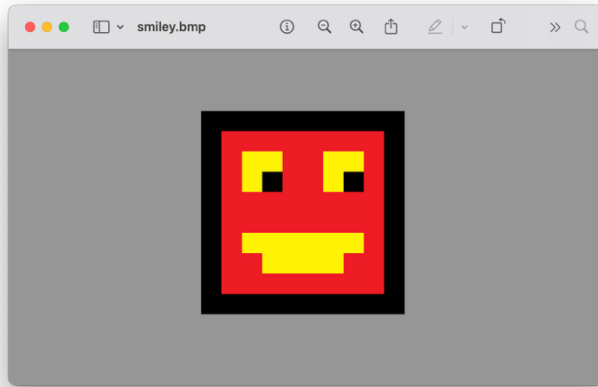
 #After it run through each line, write a next line in the "output.txt" file

```
    output_file.write("\n") #\ = Escape, \\ for \
```

#Close that output file

```
output_file.close()
```

smiley.bmp



This is a 10px * 10px bmp image file. (The image above is not the actual size)

output.txt

```
B B B B B B B B B B
B R R R R R R R R B
B R Y Y R R Y Y R B
B R Y B R R Y B R B
B R R R R R R R R B
B R R R R R R R R B
B R Y Y Y Y Y Y R B
B R R Y Y Y Y R R B
B R R R R R R R R B
B B B B B B B B B B
```

The output text file.

Woo_Matthew_Assignment1_q4.py

```
#Import image module from the pillow library
from PIL import Image

#A function called color that convert the string to RGB value
def convert(color):
    if color == "R":
        return (237, 28, 36)
    elif color == "B":
        return (0, 0, 0)
    elif color == "Y":
        return (255, 242, 0)
    else:
        return color

#Open "input.txt" in r(read) mode
input_text_file = open("input.txt", "r")

lines = input_text_file.readlines()

width, height = lines[0].count(" "), len(lines)

output_image_file = Image.new("RGB", (width, height))

#A forloop statement that go through every line
for y in range(height):
    #Assign the current line to the variable "line"
    line = lines[y]
    #Split each line to lists
    pixels = line.split()
    #A forloop statement that go through the pixel of each line
    for x in range(width):
        #Call the "convert" function to convert the string to RGB
        pixel = convert(pixels[x])
        #Put the pixels into the output image file
        output_image_file.putpixel((x,y), pixel)

#Save the image as "output.png"
output_image_file.save("output.png")

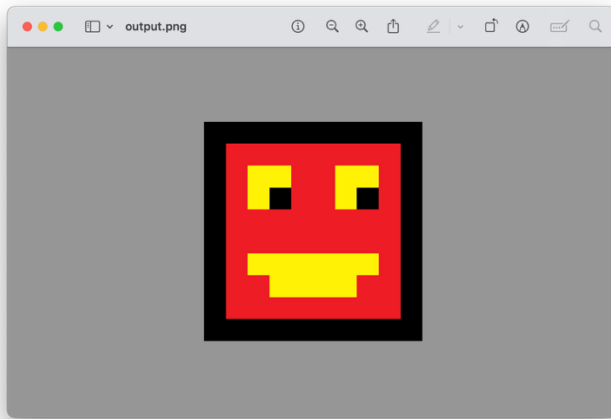
#Close the input text and output image file
input_text_file.close()
output_image_file.close()
```

input.txt

```
B B B B B B B B B B
B R R R R R R R R B
B R Y Y R R Y Y R B
B R Y B R R Y B R B
B R R R R R R R R B
B R R R R R R R R B
B R Y Y Y Y Y Y R B
B R R Y Y Y Y R R B
B R R R R R R R R B
B B B B B B B B B B
```

The input text file.

output.png



This is a 10px * 10px png image file. (The image above is not the actual size)